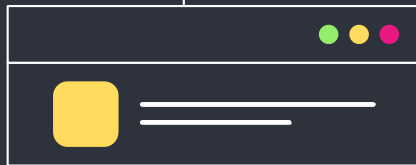


</

Estructuras de Datos con C++

/>



Nómada Project

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ ¿Qué son las Estructuras de Datos?

Son herramientas fundamentales para organizar, almacenar y manipular datos de manera eficiente. Una estructura de datos define una forma particular de organizar los datos en la memoria para que puedan ser accedidos y modificados eficientemente según el propósito del programa.

Características principales:

1. **Organización:** Los datos se organizan de manera lógica, lo que facilita su procesamiento.
2. **Eficiencia:** Ayudan a realizar operaciones como inserción, eliminación, búsqueda y actualización de manera más rápida.
3. **Flexibilidad:** Algunas estructuras son dinámicas y pueden ajustarse en tiempo de ejecución.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Clasificación de las Estructuras de Datos:

1. **Estructuras de Datos Lineales:** Los elementos se organizan secuencialmente.

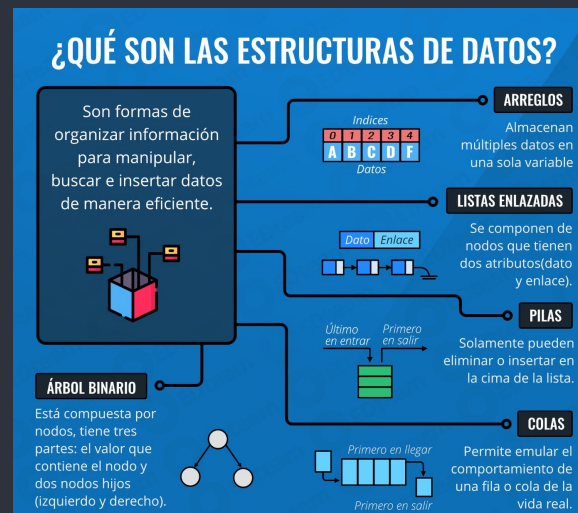
- **Arreglos:** Colecciones de elementos del mismo tipo almacenados en ubicaciones contiguas de memoria.
- **Listas enlazadas:** Secuencia de nodos donde cada nodo contiene un dato y una referencia al siguiente.
- **Pilas (Stacks):** Estructura LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir.
- **Colas (Queues):** Estructura FIFO (First In, First Out), donde el primer elemento en entrar es el primero en salir.

2. **Estructuras de Datos No Lineales:** Los datos no se organizan en una secuencia lineal.

- **Árboles (Trees):** Estructuras jerárquicas con nodos organizados en niveles, con un nodo raíz y nodos hijos.
- **Grafos (Graphs):** Conjuntos de nodos conectados mediante aristas, utilizados para modelar relaciones complejas.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

3. **Tablas Hash (Hash Tables):** Usan una función hash para mapear claves a índices en una tabla para búsquedas rápidas.
4. **Estructuras de Datos Compuestas:** Los datos no se organizan en una secuencia lineal.
 - **Estructuras:** Conjuntos de datos heterogéneos definidos por el programador.
 - **Clases y Objetos:** Estructuras avanzadas utilizadas en la programación orientada a objetos.



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</1. Recursividad

Concepto:

La recursividad es una técnica en la cual una función se llama a sí misma para resolver un problema dividiéndolo en subproblemas más pequeños y similares al problema original.

Importancia en la resolución de problemas:

- **Divide y conquista:** Facilita la resolución de problemas complejos al dividirlos en problemas más manejables.
- **Claridad y simplicidad:** Hace que algunos algoritmos sean más fáciles de entender y escribir, como en el caso del cálculo de factoriales, series de Fibonacci y recorrido de árboles.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Estructura de una función recursiva:

Una función recursiva consta de dos partes principales:

1. Caso base:

El caso base es la condición que detiene las llamadas recursivas. Representa el problema más simple que puede resolverse directamente, sin más llamadas recursivas.

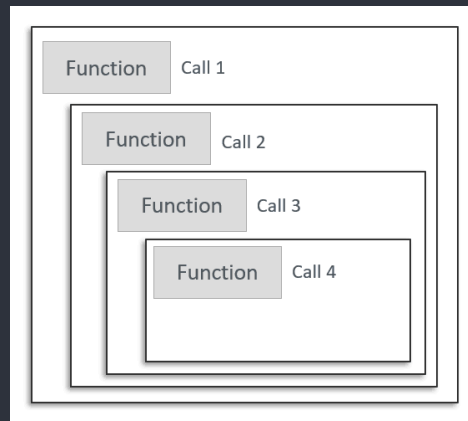
- **Propósito:** Prevenir una recursión infinita y garantizar que el algoritmo termine.
- **Identificación:** Es la condición más sencilla del problema, usualmente el caso trivial o borde como $n = 0$. Puedes preguntarte: ¿Cuál es la condición más sencilla para detener la recursión?
- **Importancia:** Sin un caso base, la recursión continuará indefinidamente, causando un error de desbordamiento de pila (stack overflow).

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

2. Caso recursivo (o caso general):

El caso recursivo es la parte del algoritmo donde la función se llama a sí misma para resolver una versión más pequeña del problema original. Es la parte que "reduce" el problema hacia el caso base.

- **Propósito:** Descomponer el problema en subproblemas más pequeños hasta que se alcance el caso base.
- **Identificación:** Se puede encontrar preguntándose: ¿Cómo puedo dividir el problema actual en algo más simple que eventualmente llegue al caso base?
- **Importancia:** Sin un caso recursivo adecuado, la recursión no logra resolver el problema.



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Comparación entre Recursión e Iteración:

Aspecto	Recursión	Iteración
Definición	Una función que se llama a sí misma.	Repetición mediante bucles.
Estructura	Caso base y llamada recursiva.	Condición de control del bucle.
Uso de memoria	Requiere espacio adicional en la pila de llamadas.	Menor uso de memoria.
Velocidad	Más lenta debido a llamadas repetidas.	Más rápida.
Claridad	Más intuitiva para problemas como Fibonacci.	Más fácil de depurar.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ejercicios:

1. Calcula el factorial de un número mediante una función recursiva.
2. Usando métodos recursivos, obtén la suma de los n primeros números naturales positivos.
3. Calcula el enésimo número Fibonacci mediante una función recursiva.
4. Usando recursividad, determina si un número es par o impar.
5. Obtén el producto de dos enteros a y b de manera recursiva.
6. Lee un número en base 10 y muestre su equivalente en base 2 por métodos recursivos.
7. Calcula la suma de los elementos de un arreglo. Usa funciones recursivas.
8. Encuentra el máximo elemento de un arreglo usando recursividad.
9. Escribe una función recursiva que invierta una cadena de texto dada.
10. Se tiene una cerradura de n dígitos, en donde cada dígito se configura independientemente del otro (como una caja Fuerte o la cerradura de un maletín). Obtenga la clave usando métodos recursivos.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1